CHAPTER 6 — SOFTWARE METRICS

Vocabulary Coverage & Master Ledger

Axis 1: Fundamentals of Measurement

Leaf 1.1: Why Measure Software — The Questions Metrics Answer

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 250–252; companion slides 2–4.
- **Coverage Question IDs:** `Q6.D01`, `Q6.DGM06`, `Exercise 6.1`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition: Software measurement answers critical management and engineering questions across four operational dimensions:**
- *Project Health:* Is the project on schedule and within budget?
- *Product Quality:* Is the system reliable enough for commercial deployment? Which components are most defect-prone?
- *Process Efficiency:* Did adopting code reviews or CI/CD reduce defect leakage?
- *Resource Optimization:* What is our baseline productivity (LOC/PM, FP/PM), and what will future projects cost? *DeMarco's Principle:* **"You cannot control what you cannot measure."**

Leaf 1.2: Measure, Measurement, Metric, and Indicator

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 251; companion slides 3–4.
- **Coverage Question IDs:** `Q6.D01`, `Q6.21`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- **Measure:** A quantitative indication of the extent, amount, dimension, capacity, or size of some attribute (e.g., raw count of 50 defects).
- **Measurement:** The formal physical or computational act of determining a measure.
- **Metric:** A quantitative measure of the degree to which a system, component, or process possesses a given attribute, typically normalizing measures (e.g., Defect Density = 50 defects / 5 KLOC = 10 defects/KLOC).
- **Indicator:** A metric or combination of metrics providing operational insight to guide technical or managerial decisions (e.g., $DRE < 80\%$ indicates an immediate need to halt deployment and audit testing gates).

Leaf 1.3: Direct versus Indirect Measures

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 252; companion slide 6; Sommerville 9e, p. 668.
- **Coverage Question IDs:** `Q6.22`, `Q6.C03`, `Q6.D06`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**

- **Direct Measures:** Quantify an attribute solely in terms of the target entity itself without calculating other variables (e.g., Lines of Code, execution runtime in milliseconds, defect counts).
- **Indirect Measures:** Quantify an attribute by mathematically combining two or more direct measures through an equation (e.g., Programmer Productivity = LOC/PM, Defect Density = $\text{Defects}/KLOC$, Module Weakness = $LV_{max} \times SP_{avg}$).

Leaf 1.4: Measurement Theory: Entities, Attributes, and Scales

- **Primary Source Anchor:** Fenton (1994), pp. 200–201; Aggarwal & Singh scan p. 252 for the course's entity/attribute examples only.
- **Coverage Question IDs:** `Q6.D05`, `Q6.21`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Fenton measurement-theory overview; Leach is not cited for scale theory)
- **Core Engineering Definition:**
- **Entity:** An object or event in the real world (e.g., source code file, design document, testing phase).
- **Attribute:** A feature or property possessed by the entity (e.g., size, complexity, reliability, execution speed).
- **Measurement Mapping:** A mapping from the empirical domain to a formal numerical system that preserves observed real-world relations.

Leaf 1.5: Scales of Measurement (Nominal, Ordinal, Interval, Ratio, Absolute)

- **Primary Source Anchor:** Stevens (1946), pp. 677–680; Fenton (1994), pp. 200–201.
- **Coverage Question IDs:** `Q6.D05`, `Q6.21`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Stevens' four-scale taxonomy; Fenton's software-measurement treatment includes absolute scale)
- **Core Engineering Definition:**
- **Nominal:** Equivalence classification into unordered categories ($=, \neq$). Permissible: Mode. (e.g., bug category).
- **Ordinal:** Ranking in order of magnitude ($>, <$). Distance unknown. Permissible: Median, rank correlation. (e.g., bug severity: High, Med, Low; CMMI 1–5).
- **Interval:** Equal intervals, arbitrary zero. Permissible: Mean, std dev. (e.g., calendar sprint dates).
- **Ratio:** Equal intervals, true non-arbitrary absolute zero. Permissible: All arithmetic, ratios, geometric mean. (e.g., LOC, execution time, defect counts).
- **Absolute:** Direct counting scale with single possible mapping. (e.g., count of developers).

Leaf 1.6: Representation Condition and Meaningfulness

- **Primary Source Anchor:** Fenton (1994), pp. 200–201.
- **Coverage Question IDs:** `Q6.D05`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Fenton 1994)
- **Core Engineering Definition:**
- **Representation Condition:** The numerical mapping must preserve all observed empirical relations.
- **Meaningfulness:** A mathematical assertion using metric values is meaningful if and only if its truth value remains invariant under all permissible scale transformations. (e.g., calculating the arithmetic mean of ordinal CMMI levels is mathematically meaningless).

Leaf 1.7: Goal–Question–Metric (GQM) Paradigm

- **Primary Source Anchor:** Leach, R., printed pp. 39–40.
- **Coverage Question IDs:** `Q6.23`, `Q6.D04`, `Q6.DGM06`.
- **Status:** `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:** Top-down measurement model:
- **Conceptual Level (Goal):** Establishes an objective for an object with a quality focus from a specific viewpoint.
- **Operational Level (Question):** Formulates questions characterizing the achievement of the goal.
- **Quantitative Level (Metric):** Identifies objective/subjective metrics to answer each question.

Leaf 1.8: Areas of Application and Implementation Problems

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 252–254; companion slides 7–8.
- **Coverage Question IDs:** `Q6.D02`, `Exercise 6.2`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- **Applications:** Effort/cost estimation, defect prevention, quality control, process improvement.
- **Problems:** Unclear objectives ("data cemeteries"), fear of punitive ranking, collection overhead; incentive-related metric gaming is treated as a possible authored scenario, not as a sourced law.

Axis 2: Categories of Metrics

Leaf 2.1: Product, Process, and Project Metrics

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 254–255; companion slides 9–10.
- **Coverage Question IDs:** `Q6.1`, `Q6.D03`, `Exercise 6.3`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- **Product Metrics:** Characteristics of the deliverables/artifacts (LOC, FP, $V(G)$, MTTF).
- **Process Metrics:** Characteristics of development activities and methods (DRE, bug fix turnaround).
- **Project Metrics:** Operational resource and schedule status (Person-Months, cost variance, schedule slip).

Leaf 2.2: Internal versus External Attributes

- **Primary Source Anchor:** Sommerville, I., 9th ed., printed pp. 668–671.
- **Coverage Question IDs:** `Q6.22`, `Q6.D06`, `Q6.C03`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Sommerville §24.4)
- **Core Engineering Definition:**
- **Internal:** Measured strictly from the artifact itself (size, cyclomatic complexity, coupling).
- **External:** Measured only with respect to runtime environment and users (reliability, maintainability, usability).

Leaf 2.3: Predictor Metrics versus Control Metrics

- **Primary Source Anchor:** Sommerville, I., 9th ed., printed pp. 668–671.
- **Coverage Question IDs:** `Q6.24`, `Q6.D06`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Sommerville §24.4)
- **Core Engineering Definition:**
- **Predictor:** Measures current internal attributes to forecast future attributes (e.g., $V(G)$ predicting defects).
- **Control:** Provides real-time visibility into an active process to maintain management control (e.g., burndown rate).

Axis 3: Size Metrics

Leaf 3.1: Lines of Code (LOC, KLOC): Definitions and Counting Rules

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–142; companion slides Ch.4.
- **Coverage Question IDs:** `Q6.D07`, `Q6.15`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Physical LOC (every line) vs Logical LOC (executable statements and data declarations). Excludes blank lines and comments in logical counting.

Leaf 3.2: Strengths and Criticisms of LOC

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–142; companion slides Ch.4 & Ch.6.
- **Coverage Question IDs:** `Q6.25`, `Q6.D07`, `Q6.C01`, `Exercise 6.20`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- **Strengths:** Intuitive, automated, massive historical baselines.
- **Criticisms:** Language dependence, Inverse Productivity Paradox, late availability, indifference to design complexity.

Leaf 3.3: Function Points: Components, Complexity Weights, and VAF

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–146; companion slides Ch.4.
- **Coverage Question IDs:** `Q6.26`, `Q6.27`, `Problem 7.10`, `Problem 7.11`, `Exercise 6.11`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- 5 components: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), External Interface Files (EIF).
- Complexity weights: Low, Average, High.

- $VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$, ranging from 0.65 to 1.35.
- $FP = UFP \times VAF$.

Leaf 3.4: Feature Points and Mark II Function Points

- **Primary Source Anchor:** Pressman 5e (PDF pp. 120–122) for Feature Points; Mark II FP is an explicit documented gap.
- **Coverage Question IDs:** `Q6.S02`, `V6_SOURCE_COVERAGE_MAP.md`.
- **Status:** Feature Points: `VERIFIED SUPPORTING SOURCE` (Pressman Ch.4); Mark II FP: `DOCUMENTED BOUNDED GAP` (Noted in `Ch06_BUILD_PLAN.md`).
- **Core Engineering Definition:**
- **Feature Points:** Extends Albrecht FP by adding a 6th component, **Algorithms**, weighted by mathematical complexity, adapting FP for real-time and systems software.
- **Mark II FP (Symons):** Replaces 5 components with Input, Process, and Output transaction flows (gap documented).

Leaf 3.5: Object Points (Application Points)

- **Primary Source Anchor:** Rajib Mall (4th ed.), printed pp. 165–166.
- **Coverage Question IDs:** `Q6.28`, `Q6.S02`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Mall §3.4)
- **Core Engineering Definition:**
- Sizes early prototypes by counting Screens, Reports, and 3GL components, classifying each by complexity. Used in COCOMO II Early Prototyping.

Leaf 3.6: Converting Between Size Measures (Backfiring)

- **Primary Source Anchor:** Pressman 5e (PDF pp. 120–122); Capers Jones (1996).
- **Coverage Question IDs:** `Q6.C01`.
- **Status:** `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Statistical conversion tables mapping Function Points to equivalent LOC based on language expansion ratios (e.g., 1 FP $\approx$ 128 C statements, $\approx$ 53 Java statements, $\approx$ 21 Python statements).

Axis 4: Halstead's Software Science

Leaf 4.1: Token Count: Operators and Operands

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–256; companion slides 11–13.
- **Coverage Question IDs:** `Q6.2`, `Q6.3`, `Problem 7.1`, `Exercise 6.10`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- η_1: Unique operators; η_2: Unique operands; N_1: Total operators; N_2: Total operands.

Leaf 4.2: Vocabulary (η) and Program Length (N)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 256; companion slides 12–13.
- **Coverage Question IDs:** `Q6.3`, `Problem 7.1`, `Problem 7.2`, `Exercise 6.7`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $\eta = \eta_1 + \eta_2$; $N = N_1 + N_2$.

Leaf 4.3: Program Volume (V) and Potential Volume (V^*)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 257; companion slides 14–16.
- **Coverage Question IDs:** `Q6.4`, `Q6.13`, `Q6.14`, `Problem 7.1`, `Problem 7.3`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $V = N \log_2 \eta$ (in bits); $V^* = (2 + \eta_2^*) \log_2 (2 + \eta_2^*)$ (minimal algorithm representation).

Leaf 4.4: Program Level (L) and Difficulty (D)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 257–259; companion slides 17–19.
- **Coverage Question IDs:** `Q6.29`, `Problem 7.1`, `Problem 7.3`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $D = \frac{1}{L} \approx \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$; $L = \frac{V^*}{V} \approx \frac{2}{\eta_1} \times \frac{\eta_2}{N_2}$.

Leaf 4.5: Effort (E) and Estimated Time (T)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 259–260; companion slides 19–20.
- **Coverage Question IDs:** `Q6.5`, `Q6.12`, `Problem 7.1`, `Problem 7.3`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $E = D \times V = \frac{V}{L}$ (in elementary mental discriminations); $T = \frac{E}{S} = \frac{E}{18}$ seconds.

Leaf 4.6: Estimated Program Length ($\hat{N}$) and Error Percentage ($\hat{B}$)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 256, 260; companion slides 13, 21.
- **Coverage Question IDs:** `Problem 7.1`, `Problem 7.2`, `Exercise 6.7`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$; $\hat{B} = \frac{V}{3000}$.

Leaf 4.7: Language Level (λ) and the Language-Level Table

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 260; companion slides 22–23.

- **Coverage Question IDs:** `Q6.6`, `Problem 7.3`, `Exercise 6.5`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $\lambda = L \times V^* = L^2 V$. Measures language expressiveness independently of program size.

Leaf 4.8: Counting Rules for C

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 262–265; companion slides 24–38.
- **Coverage Question IDs:** `Q6.D09`, `Exercise 6.10`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Paired delimiters count as 1 operator; function calls count as operators; comments and `#include` excluded.

Leaf 4.9: Criticisms and Empirical Validity of Software Science

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 261–262; companion slide 23.
- **Coverage Question IDs:** `Q6.D10`, `Q6.C02`, `Exercise 6.4`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- May correlate with LOC because token counts are taken from source code; no numeric correlation or universal predictive limitation is asserted from the inspected course source. Other criticisms require their own supporting evidence.

Axis 5: Data Structure Metrics

Leaf 5.1: Input, Internal, and Output Data

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 273–274; companion slide 39.
- **Coverage Question IDs:** `Q6.D11`, `Exercise 6.12`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Input: data passed from caller/environment; Internal: local temporary variables; Output: data returned or exported.

Leaf 5.2: Amount of Data: $VARS$, η_2, Cross-Reference Lists

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 274–275; companion slides 41–44.
- **Coverage Question IDs:** `Q6.D11`, `Q6.DGM03`, `Exercise 6.14`.
- **Status:** `VERIFIED COURSE SOURCE`

- **Core Engineering Definition:**
- $VARS$: distinct variables count; cross-reference lists index declaration, assignment, and usage statements.

Leaf 5.3: Live Variables (LV): Three Definitions

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 275–277; companion slides 45–53.
- **Coverage Question IDs:** `Q6.D11`, `Q6.DGM04`, `Problem 7.4` (authored trace).
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- A variable is live between its definition statement and its final reference statement without intervening death.

Leaf 5.4: Average Live Variables ($\bar{LV}$)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 277; companion slide 53.
- **Coverage Question IDs:** `Q6.D11`, `Q6.DGM04`, `Problem 7.4`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $\bar{LV} = \frac{1}{n}\sum_{i=1}^{n} LV(S_i)$ across all n executable statements.

Leaf 5.5: Variable Span (SP) and Average Span Size (SP_{avg})

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 277–279; companion slides 54–62.
- **Coverage Question IDs:** `Q6.D11`, `Q6.DGM04`, `Problem 7.4`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Intervening statements between successive references: $SP = b - a - 1$; $SP_{avg} = \frac{\sum SP}{\text{Total Spans}}$.

Leaf 5.6: Module Weakness (WM) and Program Weakness (WP)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 278–280; companion slides 63–64, 68.
- **Coverage Question IDs:** `Q6.7` (Defect callout), `Q6.D11`, `Problem 7.4`, `Problem 7.5`, `Exercise 6.13`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $WM = \overline{LV} \times \gamma$; $WP = \frac{1}{m}\sum_{i=1}^{m} WM_i$, where $\overline{LV} = (\sum LV)/n$ (average live variables per statement) and $\gamma = (\sum LV)/\text{total variables}$ (average variable life).

Leaf 5.7: Deriving Program-Wide Metrics from Module Metrics

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 280; companion slides 63–68.
- **Coverage Question IDs:** `Problem 7.5`, `Exercise 6.13`.

- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Aggregating modular metrics using arithmetic means, identifying variance and structural outliers across components.

Axis 6: Information Flow Metrics

Leaf 6.1: Fan-in and Fan-out

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 282; companion slides 69–70.
- **Coverage Question IDs:** `Q6.8`, `Q6.9`, `Problem 7.6`, `Problem 7.7`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Fan-in: Calling modules + data structures read; Fan-out: Called modules + data structures updated.

Leaf 6.2: Henry and Kafura's Information Flow Metric (HKM)

- **Primary Source Anchor:** Pressman 5e §19.4.2, printed p. 524 / PDF p. 552; Aggarwal companion slide 71.
- **Coverage Question IDs:** `Q6.30`, `Q6.D12`, `Problem 7.7`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Pressman 5e §19.4.2)
- **Core Engineering Definition:**
- $HKM = \text{length}(i) \times [fan_in(i) + fan_out(i)]^2$.

Leaf 6.3: Length and Complexity Weighting

- **Primary Source Anchor:** Pressman 5e §19.4.2; Aggarwal slides 71–75.
- **Coverage Question IDs:** `Q6.D12`, `Problem 7.7`.
- **Status:** `VERIFIED COURSE SOURCE` & `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Module length in LOC provides linear scaling; the squared sum factor models exponential interface channel complexity.

Leaf 6.4: IF4 and Shepperd's Refinements

- **Primary Source Anchor:** Unverified in current local textbook scans.
- **Coverage Question IDs:** `Ch06_BUILD_PLAN.md`, `V6_SOURCE_COVERAGE_MAP.md`.
- **Status:** `DOCUMENTED BOUNDED GAP` (Preserved without ungrounded derivation).
- **Core Engineering Definition:**
- Refinements to remove duplicate flow counting and isolate control flow from data flow (documented gap).

Leaf 6.5: Interpreting High Fan-out and High Fan-in

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 74–77.
- **Coverage Question IDs:** `Q6.S03`, `Slide 74`.
- **Status:** `VERIFIED COURSE SOURCE`

- **Core Engineering Definition:**
- High Fan-in: High reuse, but single point of failure. High Fan-out: Over-coordination, God Object risk.

Axis 7: Control Flow and Complexity Metrics

Leaf 7.1: Cyclomatic Complexity (McCabe): Three Computation Methods

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–425; companion slides Ch.8.
- **Coverage Question IDs:** `Q6.10`, `Q6.31`, `Q6.D13`, `Q6.DGM01`, `Q6.DGM02`, `Problem 7.8`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $V(G) = E - N + 2P = P_{pred} + 1 = \text{Planar Regions}$.

Leaf 7.2: Essential Complexity and Design Complexity

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–425 (cyclomatic complexity only).
- **Coverage Question IDs:** `Q6.D13`.
- **Status:** `DOCUMENTED BOUNDED GAP`
- **Core Engineering Definition:**
- Essential Complexity and Design Complexity are requested terms, but definitions and computation rules were not verified in the approved local source set inspected for this pack.

Leaf 7.3: Knot Count and Nesting Depth

- **Primary Source Anchor:** No verified local source anchor found in the approved sources inspected for this pack.
- **Coverage Question IDs:** `Q6.D14`.
- **Status:** `DOCUMENTED BOUNDED GAP`
- The requested definitions, computation rules, and any numeric cutoff remain unverified; none is asserted here.

Leaf 7.4: Relationship Between Complexity and Testability

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 424–425.
- **Coverage Question IDs:** `Q6.D13`, `Q6.C02`, `Q6.S04`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- $V(G)$ defines the cardinality of the linearly independent basis path set, giving the number of paths in an independent basis; this is not by itself a claim of complete branch or condition coverage.

Axis 8: Object-Oriented Metrics

Leaf 8.1: Why Traditional Metrics Fall Short for OO Systems

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–289; companion slides 78–79.
- **Coverage Question IDs:** `Q6.D15`, `Q6.C05`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Small method paradox ($V(G) = 1$ for short methods); complexity shifts to inheritance, dynamic binding, and message passing.

Leaf 8.2: The Chidamber & Kemerer (CK) Suite (WMC, DIT, NOC, CBO, RFC, LCOM)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–293; companion slides 80–85.
- **Coverage Question IDs:** `Q6.16`, `Q6.18`, `Q6.32`, `Q6.D15`, `Q6.C06`, `Q6.S05`, `Problem 7.12`, `Exercise 6.27`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- WMC: Method weight sum; DIT: Max root path; NOC: Direct subclasses; CBO: Coupled classes; RFC: Response set; LCOM: $P - Q$ (dissimilarity).

Leaf 8.3: MOOD Metrics Suite

- **Primary Source Anchor:** Pressman 5e (PDF pp. 686–695); companion slide 88.
- **Coverage Question IDs:** `Q6.D16`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Pressman Ch.24)
- **Core Engineering Definition:**
- Method/Attribute Inheritance Factors (MIF, AIF), Hiding Factors (MHF, AHF), Coupling Factor (CF), Polymorphism Factor (PF).

Leaf 8.4: Lorenz and Kidd Metrics

- **Primary Source Anchor:** Pressman 5e (PDF pp. 686–695); companion slide 88.
- **Coverage Question IDs:** `Q6.D16`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Pressman Ch.24)
- **Core Engineering Definition:**
- Class size, method size, and inheritance metrics quantifying pragmatic OO implementation properties.

Leaf 8.5: Class Size, Method Size, and Cohesion (TCC, LCC)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 292–293; companion slides 85–86.
- **Coverage Question IDs:** `Q6.17`, `Q6.C07`.
- **Status:** `VERIFIED COURSE SOURCE`

- **Core Engineering Definition:**
- Tight Class Cohesion (TCC): Direct shared attribute access; Loose Class Cohesion (LCC): Direct + indirect accesses ($TCC \leq LCC$).

Axis 9: Use-Case Oriented Metrics

Leaf 9.1: Use Case Points (UCP)

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (printed pp. 295–296; companion slides 89–90).
- **Coverage Question IDs:** `Q6.D17`, `Problem 7.13`.
- **Status:** `PARTIALLY VERIFIED COURSE SOURCE; ADJUSTMENT/EFFORT EXTENSIONS DOCUMENTED AS GAPS`
- **Core Engineering Definition:**
- The available course material supports the unadjusted measure $UUCP = UAW + UUCW$. A complete adjusted UCP procedure is not established by the approved local source set inspected for this pack.

Leaf 9.2: Unadjusted Actor Weights and Use Case Weights

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh companion slides 89–90.
- **Coverage Question IDs:** `Q6.D17`, `Problem 7.13`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Actors: Simple (1), Average (2), Complex (3); Use Cases: Simple (≤ 3, wt 5), Average (4–7, wt 10), Complex (≥ 8, wt 15).

Leaf 9.3: Technical and Environmental Complexity Factors

- **Status:** `DOCUMENTED BOUNDED GAP`
- The supplied course source pages/slides inspected here do not give a verifiable TCF/ECF formula, factor list, or weights. Do not calculate adjusted UCP from this pack.

Leaf 9.4: Estimating Effort from Use Case Points

- **Status:** `DOCUMENTED BOUNDED GAP`
- No effort-per-UCP conversion was verified in the approved local sources inspected for this pack. Do not apply a generic hours/UCP constant as if it were part of the course material.

Leaf 9.5: Comparison with Function Points

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh, Chapter 4 Function Point discussion and Chapter 6 UCP slides 89–90.
- **Coverage Question IDs:** `Q6.D17`.
- **Status:** `PARTIALLY VERIFIED COURSE-SOURCE COMPARISON`
- **Core Engineering Definition:**

- The course material presents use-case sizing through actors/use cases and function-point sizing through functional components. No stronger comparative claim is made here.

Axis 10: Quality and Defect Metrics

Leaf 10.1: Defect Density and Defect Removal Efficiency (DRE)

- **Primary Source Anchor:** Pressman 5e, printed p. 98 / PDF p. 126; companion slide 10.
- **Coverage Question IDs:** `Q6.34`, `Q6.D18`, `Q6.C10`, `Problem 7.14`.
- **Status:** `VERIFIED SUPPORTING SOURCE` (Pressman §4.3)
- **Core Engineering Definition:**
- $\text{Defect Density} = \frac{\text{Defects}}{KLOC}$; $DRE = \frac{E}{E+D}$.

Leaf 10.2: Phase Containment Effectiveness (PCE)

- **Primary Source Anchor:** Mall, R. (4th ed.), printed pp. 84–85; Pressman 5e.
- **Coverage Question IDs:** `Q6.D18`, `Problem 7.15`.
- **Status:** `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Percentage of defects caught within the exact phase they were introduced.

Leaf 10.3: Mean Time Between Failures (MTBF) as a Product Metric

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), Ch.7, printed p. 308.
- **Coverage Question IDs:** `Q6.D18`.
- **Status:** `VERIFIED COURSE SOURCE` (Aggarwal Ch.7)
- **Core Engineering Definition:**
- $MTBF = MTTF + MTTR$. Statistical uptime reliability.

Leaf 10.4: Customer-Reported Problem Metrics

- **Primary Source Anchor:** Pressman 5e, printed p. 126.
- **Coverage Question IDs:** `Q6.D18`, `Problem 7.14`.
- **Status:** `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Defect count and Mean Time to Defect Resolution reported by post-deployment users.

Axis 11: Productivity and Process Metrics

Leaf 11.1: Productivity Measures: LOC/PM and FP/PM

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–146; companion slides Ch.4.

- **Coverage Question IDs:** `Q6.25`, `Q6.C01`, `Problem 7.15`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Physical vs functional productivity rates normalized per person-month of engineering effort.

Leaf 11.2: Effort Distribution Across Phases

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 147; companion slides Ch.4.
- **Coverage Question IDs:** `Q6.D03`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Empirical 40-20-40 rule (Requirements/Design 40%, Coding 20%, Testing 40%).

Leaf 11.3: Rework and Cost of Quality (CoQ)

- **Primary Source Anchor:** Pressman 5e, printed pp. 94–96.
- **Coverage Question IDs:** `Q6.D02`.
- **Status:** `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Prevention costs, appraisal costs, internal failure costs, and external failure costs.

Leaf 11.4: Process Maturity Metrics

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 254; companion slide 10.
- **Coverage Question IDs:** `Q6.D03`, `Exercise 6.19`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- CMMI maturity levels (1 to 5) tracking organizational process institutionalization.

Axis 12: Metrics Analysis, Program Governance & Ethics

Leaf 12.1: Statistical Analysis: Correlation, Regression, and Outliers

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 296–300; companion slides 92–95.
- **Coverage Question IDs:** `Q6.S07`, `Exercise 6.18`.
- **Status:** `VERIFIED COURSE SOURCE`
- **Core Engineering Definition:**
- Bivariate correlation (r), multivariate regression, outlier detection; caution: correlation is not causation.

Leaf 12.2: Theoretical and Empirical Validation of Metrics

- **Primary Source Anchor:** No exact approved-source page verified for the requested theoretical-versus-empirical validation distinction.
- **Coverage Question IDs:** No dedicated question; `Q6.D05` covers scale theory, not metric validation.

- **Status:** `PARTIAL — validation-method source/question remains open`
- **Core Engineering Definition:**
- Validation methods are not developed in this pack. Keep this as an explicit uncovered term pending a source-backed addition.

Leaf 12.3: Establishing a Metrics Program, Baselines, and Tooling

- **Primary Source Anchor:** K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 7–8; Pressman 5e, pp. 127–135.
- **Coverage Question IDs:** `Q6.D02`, `Q6.S06`.
- **Status:** `VERIFIED COURSE SOURCE` & `VERIFIED SUPPORTING SOURCE`
- **Core Engineering Definition:**
- Continuous data collection automated via CI/CD, creating historical organizational baselines.

Leaf 12.4: Common Pitfalls: Measurement Dysfunction and Metric Gaming

- **Primary Source Anchor:** Sommerville, 9th ed., §1.2, printed pp. 14–16, for professional duties; metric-target scenarios are authored illustrations.
- **Coverage Question IDs:** `Q6.35`, `Q6.D19`, `Q6.S06`.
- **Status:** `PARTIAL — professional duties verified; metric-gaming mechanisms are illustrative, not attributed as a sourced law`
- **Core Engineering Definition:**
- A single incentive target can encourage optimizing the measured number over the intended outcome; this is a possible scenario risk, not a sourced quotation or guaranteed effect.

Leaf 12.5: Ethics of Software Measurement

- **Primary Source Anchor:** Sommerville, 9th ed., §1.2, printed pp. 14–16, for the professional code; application to measurement practice is an authored synthesis.
- **Coverage Question IDs:** `Q6.D19`, `Q6.S06`.
- **Status:** `PARTIAL — professional code verified; no universal ban on individual-level metrics is claimed`
- **Core Engineering Definition:**
- Apply professional judgment and consider public interest, product quality, and obligations to colleagues; do not treat one metric as a complete assessment of a person or product.

- Review the status tags across all 12 axes and explain why Mark II FP and IF4/Shepperd are labeled as documented bounded gaps.
- State which primary chapter from Aggarwal provides the Cyclomatic Complexity formula and why it is cross-referenced from Chapter 8.
- Explain the difference between Stevens' interval scale and ratio scale as applied to software sizing.
- Describe a possible risk when a narrow metric becomes an incentive target, and distinguish a scenario illustration from a sourced claim.